

面向团队协作的 Agent Insight





尚卓燃
ASF Member
Senior Engineer @ OceanBase
GitHub: PsiACE



张慧
CNCF Ambassador
DevRel @ OceanBase



Content

目录

01 Agent 来了，然后呢？

02 bub，先让 Agent 学会协作

03 bubseek，让 Agent 的足迹变成团队洞察

04 让一切自然发生



Part 01

Agent 来了，然后呢？



到处都是 Agent ， 然后呢？

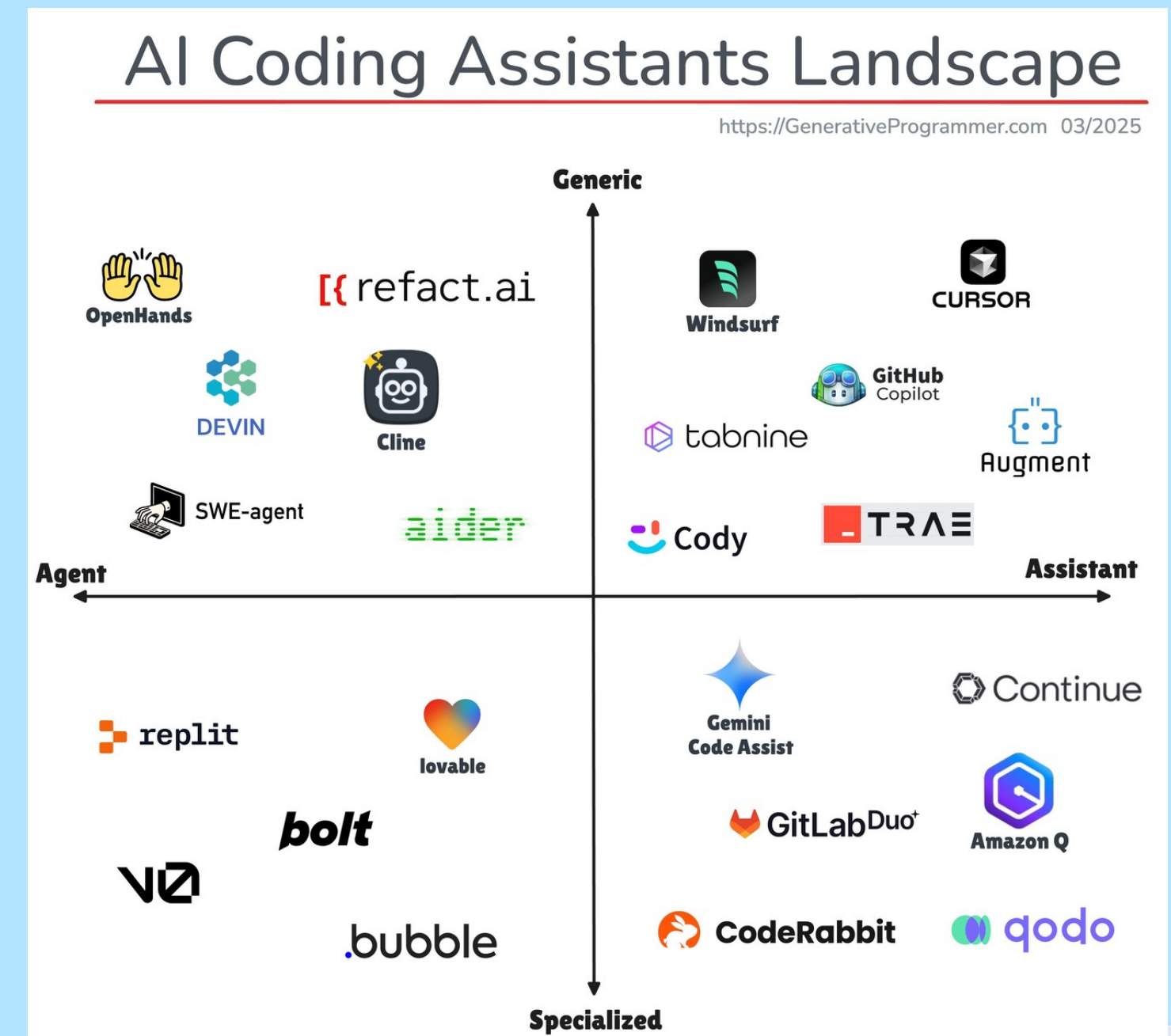
Agent 不只是工具调用器，它正在进入真实协作现场。

- 开发者让 Agent 写代码、补测试、查上下文；
- 运维让 Agent 看日志、排查异常、辅助处理告警；
- 在群聊，也有能回答问题、分发任务、调用工具的 Agent。

当 Agent 真的进入团队、进入项目、进入日常协作之后，

- 我们到底该怎么理解它？

图源：<https://generativeprogrammer.com/p/ai-coding-assistants-landscapet>
(2025.03)



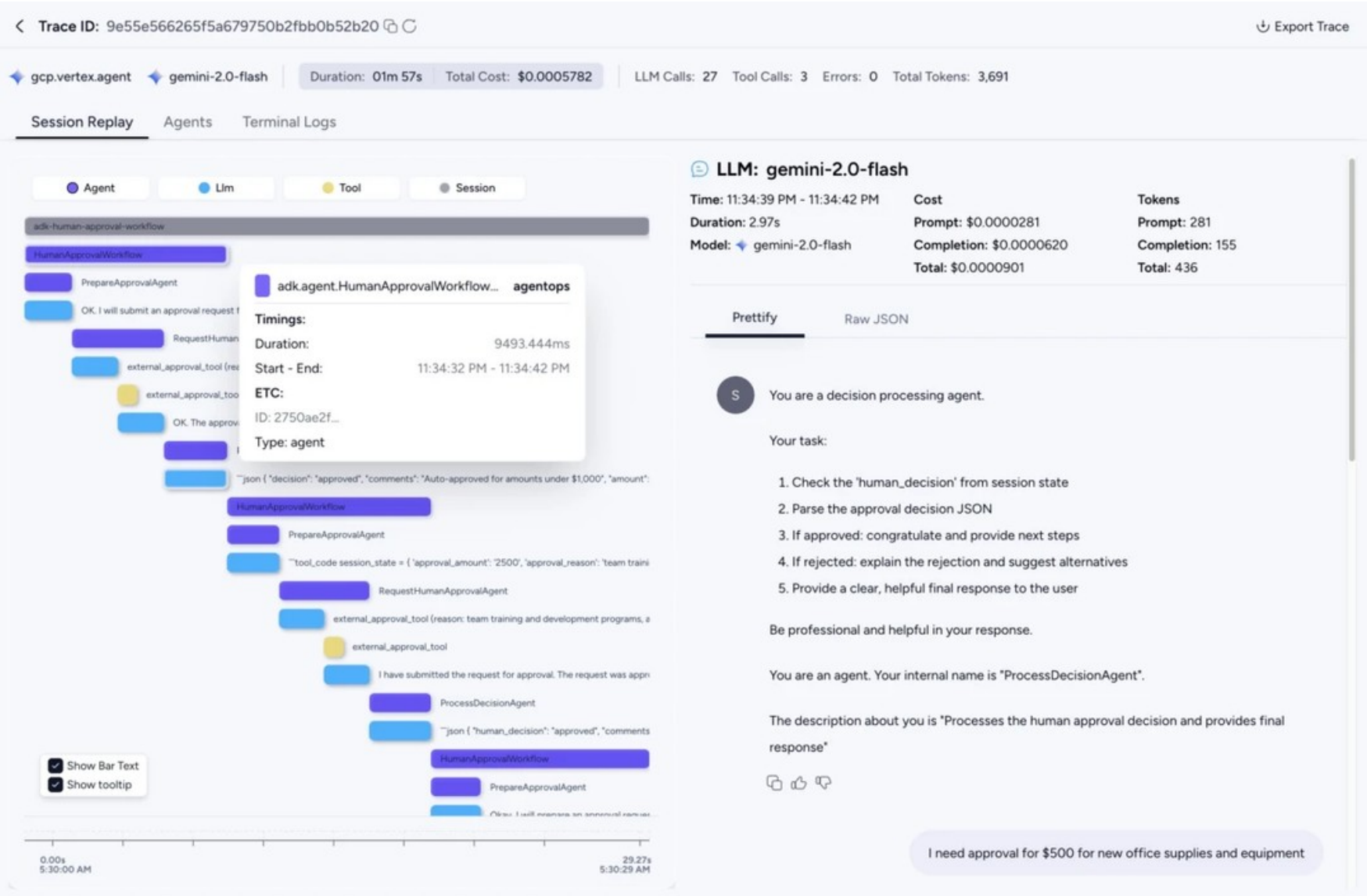
Agent 真的理解系统吗？

Agent 的表现已经足够“能干”。

- 会执行，不等于会理解；
- 能完成任务，也不等于能留下上下文。

我们需要 Agent 回答以下几类问题：

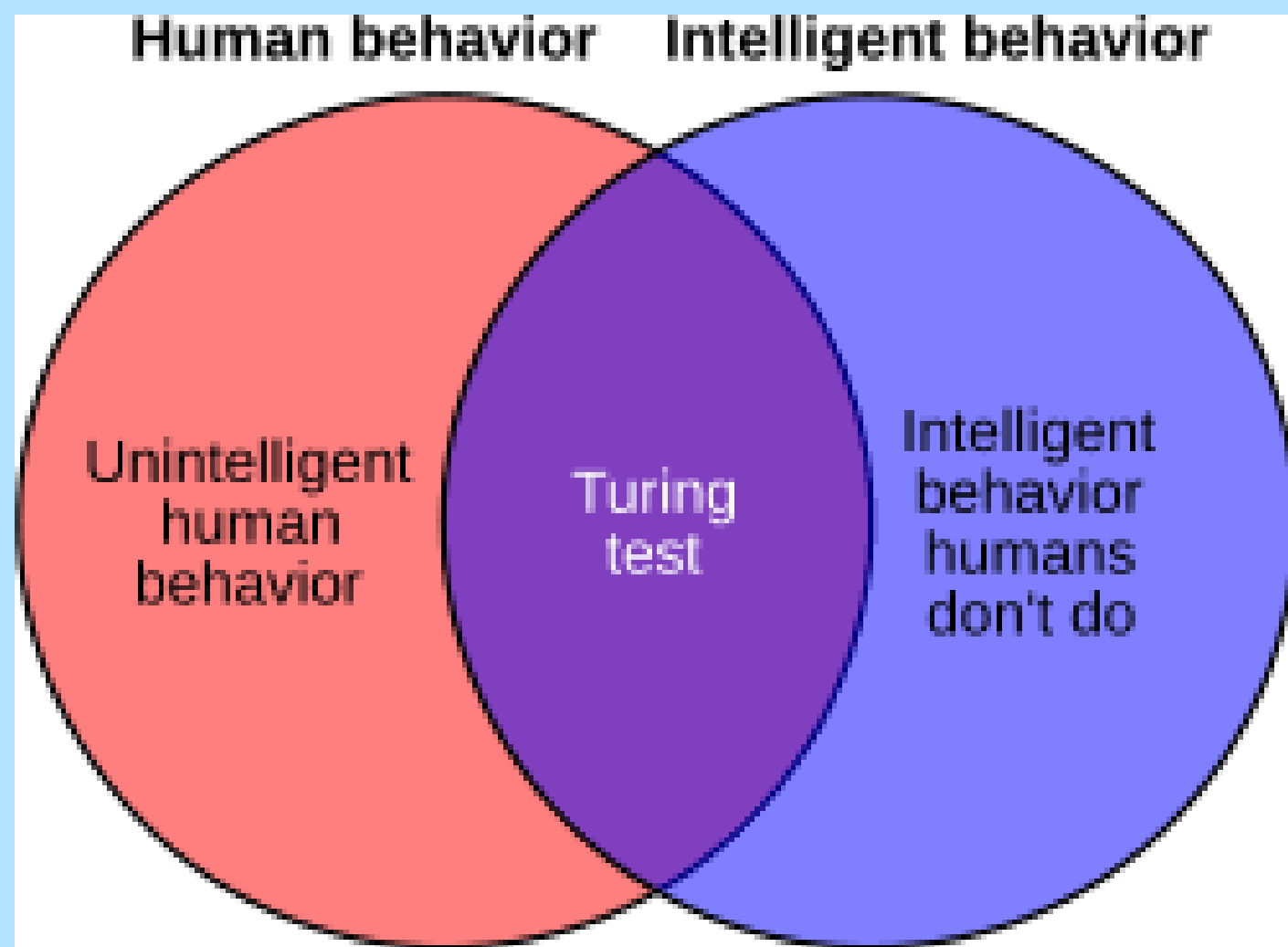
- 这个模块为什么总出问题？
- 为什么最近总在重复处理同一类故障？
- 团队这段时间到底在忙什么？
- 哪些需求被反复提到，但一直没有被很好满足？



图源：agentops.ai，过去我们为可观测性搭建了各种平台，但是这并不意味着理解



不只是“更强的 Agent”



如果 Agent 只是偶尔被调用一次，
那它像工具也没关系。

当 Agent 成为团队中常驻的一份子，
它首先是协作者，而不是工具。

那问题就不再是“它能不能做事”，而是：
能不能好好协作。

人和 Agent 要共享一套协作方式。



Part 02 bub

先让 Agent 学会协作

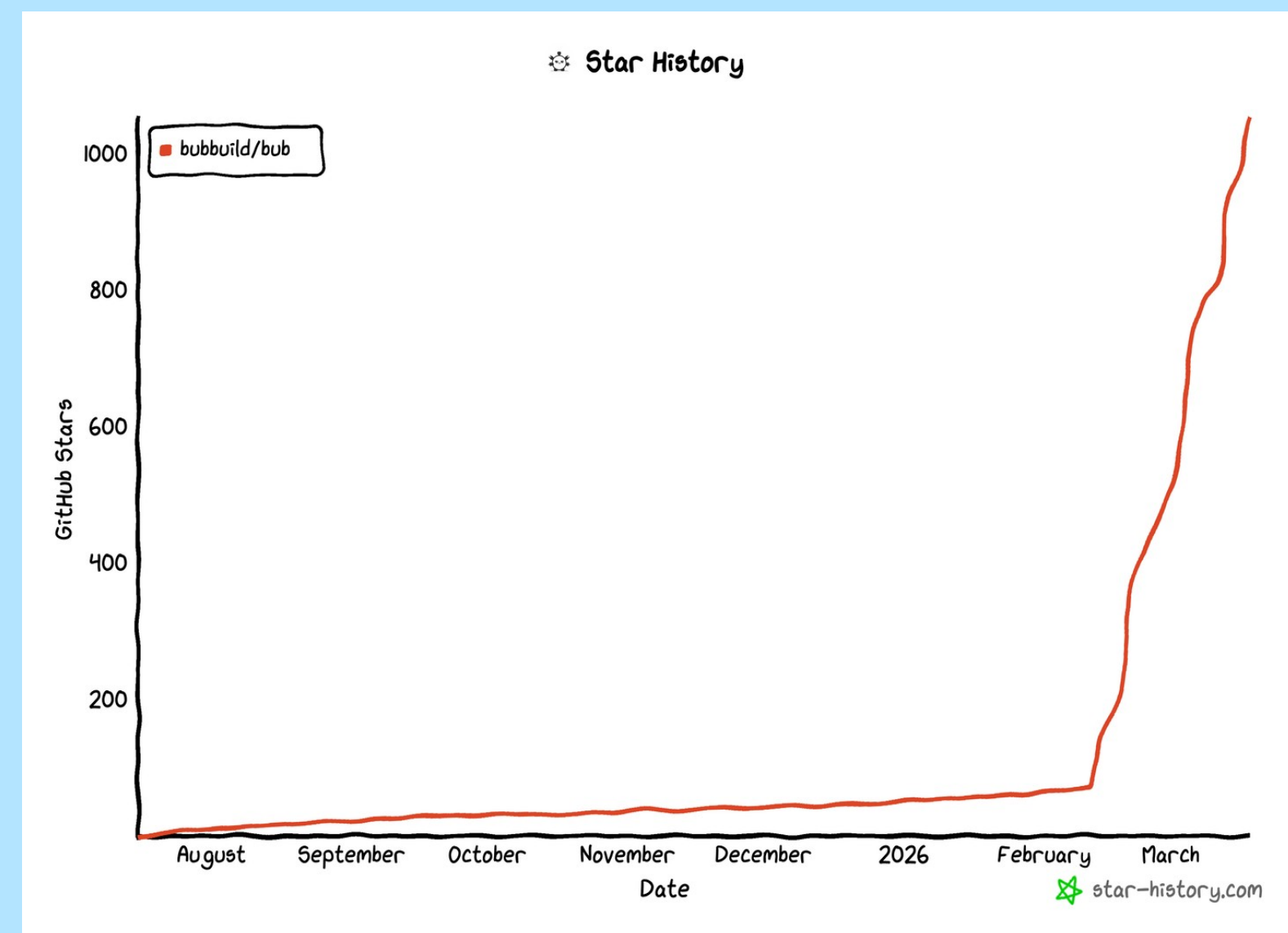


给 Agent 划边界

真正进入团队之后，
最先暴露的问题，往往不是能力不够，
而是它没有一个清晰的协作形状。

交互的中心，
不是 Agent，而是 turn。
每一次交互，是一个有边界、有阶段、可以被接管的过程。

Agent 只是这个过程里的一个环节。



图源：<https://www.star-history.com> (2026.03)

Agent → Turn

bub 关心的不是“某个 Agent 类有多强”，
而是“一次 agent turn 怎样被拆成清晰、可接管的阶段”。

- 一次 agent turn 被拆成明确阶段 (session / state / prompt / model / output)
- 每个阶段都有清晰职责，而不是混在一起
- 系统通过 hook 暴露这些阶段
- 不同能力在不同阶段接入，而不是堆在一个 Agent 里
- 边界来自结构，而不是权限控制

Traditional Agent Systems	Bub
Agent is core	Turn is core
Black-box execution	Structured lifecycle
Hidden reasoning	Tape (evidence)
Stateless runs	Anchor / handoff



给 Agent 留证据

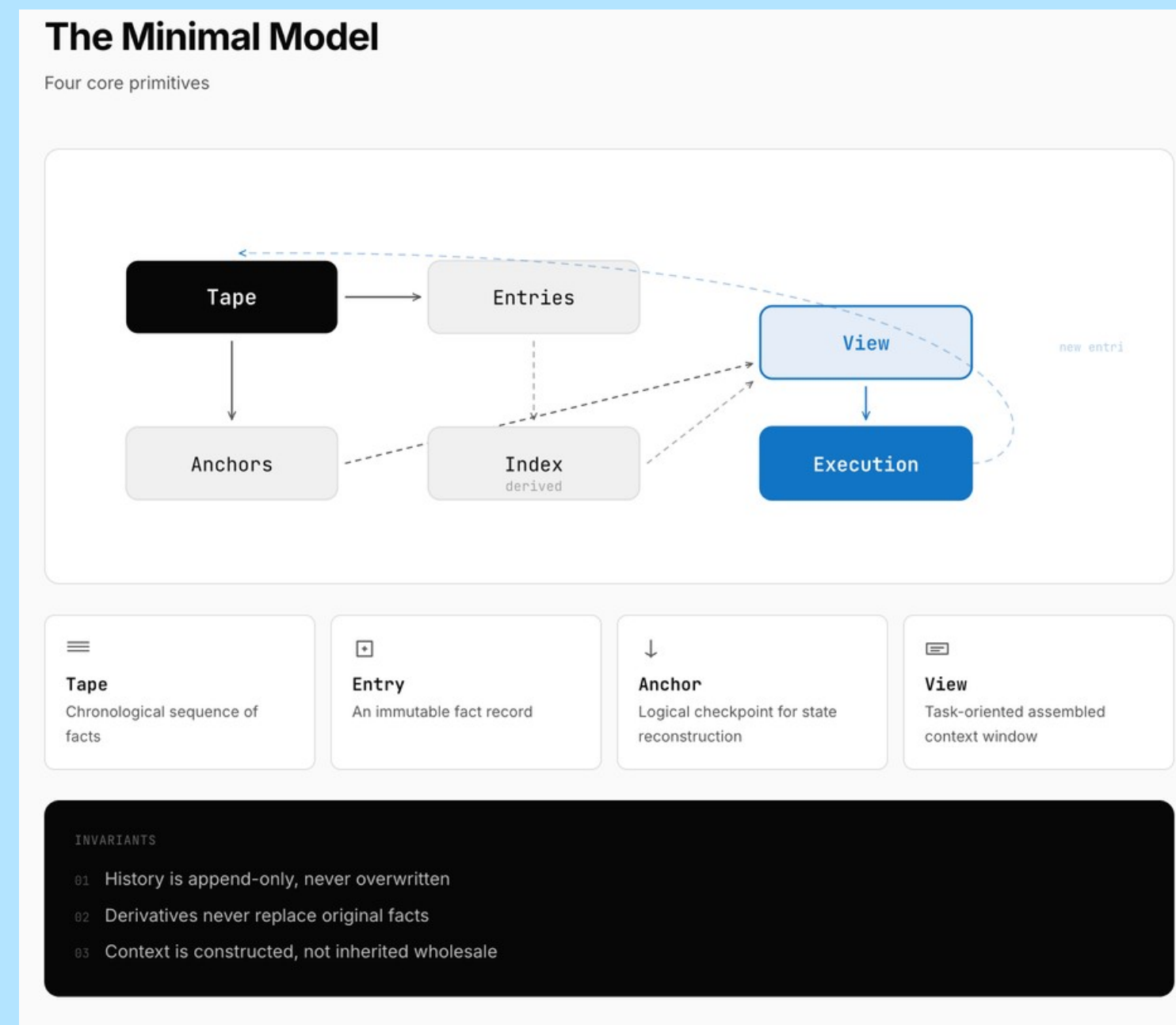
bub 用 Tape 记录一次 turn 里的关键事实，
让行为可以被复盘、检查和复用。

Tape 不是简单的聊天记录，
它记录的是一次 Agent 运行过程中的关键事实，
它不仅承载可观测性，也是 Bub 的上下文模型。

出了问题可以复盘。

做对了可以复用。

人能真正参与检查。



图源：<https://tape.systems> (2026.03)

让 Agent 可以交接

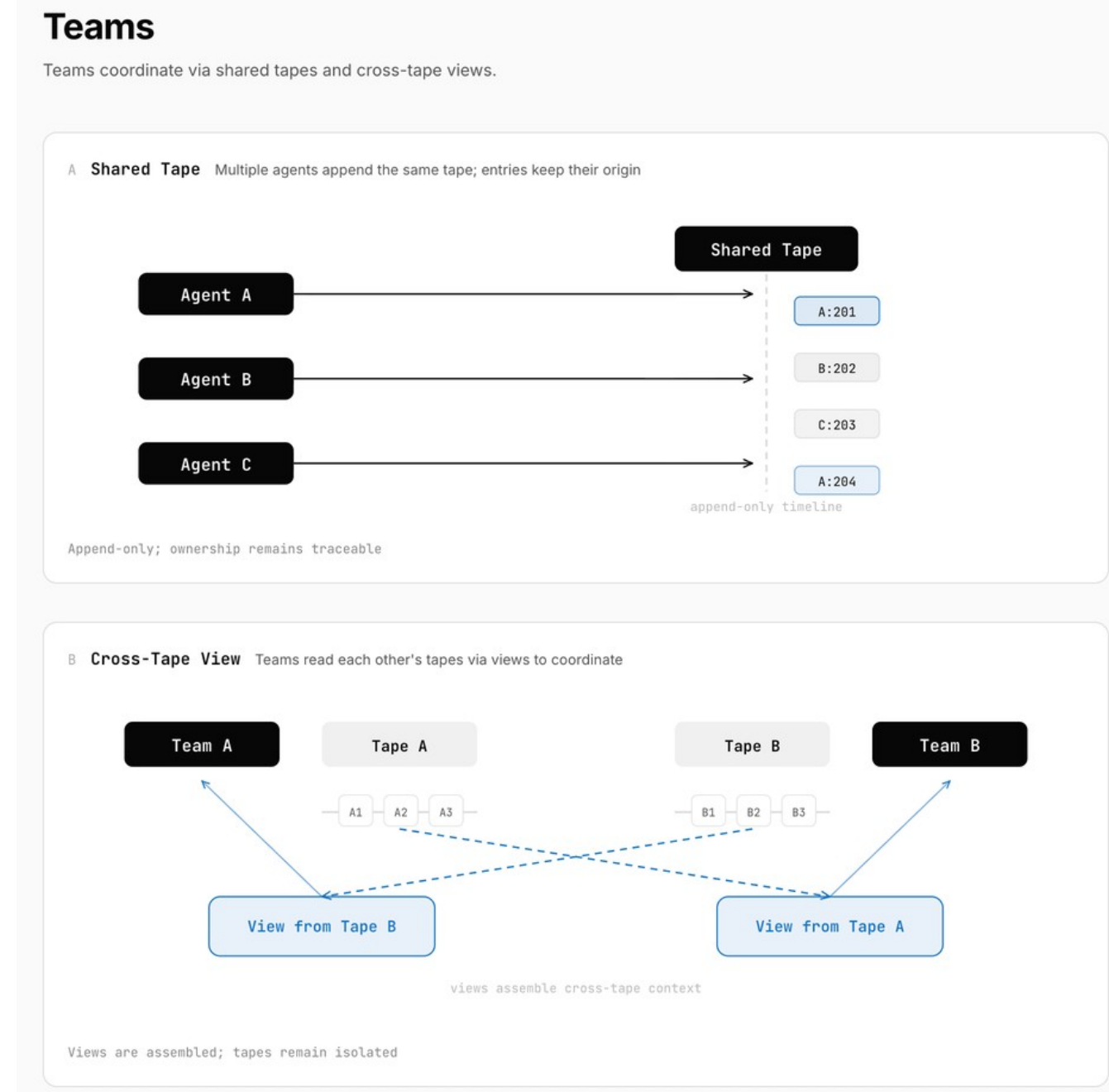
真实世界里的任务，很少是一口气做完的。

在 bub 里，交接不是补充功能，
而是一开始就放进设计里的事情。

Agent 的工作不再只是“这次回答出来就结束”，
而是可以自然地嵌入长期协作。

很多系统只关心“现在能不能把事情做完”，
而 bub 还关心：

下一位操作者，能不能在这个基础上继续向前。



图源：<https://tape.systems> (2026.03)

Part 03 **bubseek**

让 Agent 的足迹变成团队洞察



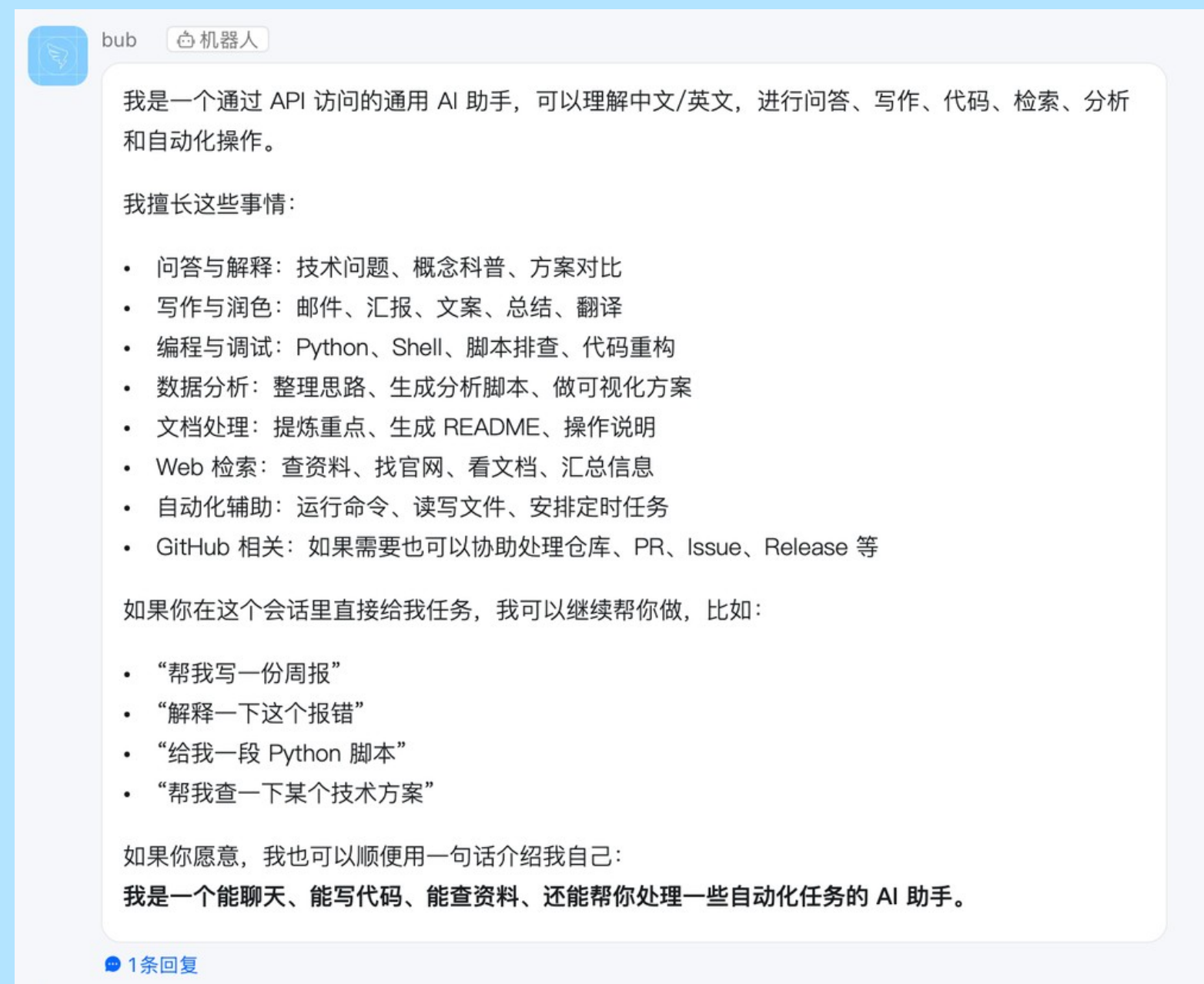
从轨迹到洞察

把 Agent 产生的轨迹，当成可分析的数据来源，
进一步转化成团队可以消费的 insight。

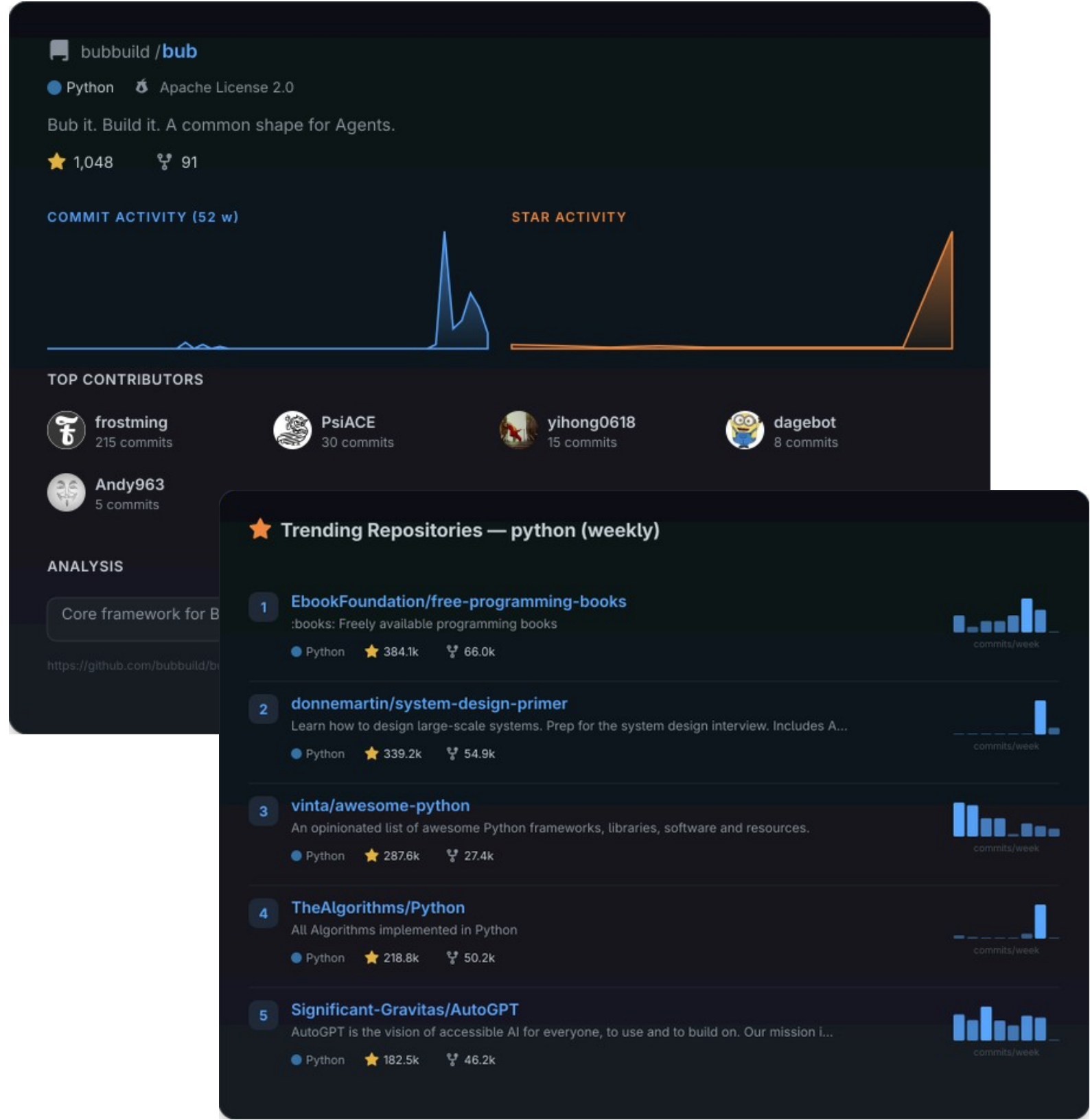
让 Agent 不只干活，还能把干活留下来的痕迹，
变成可读、可问、可分享的洞察。

bubseek 是一个建在 bub 和 seekdb 之上的
自驱动洞察 Agent。

图源：钉钉中的 bubseek，由 codex 驱动（2026.03）



把需求变成一句话



企业内部的数据需求，
通常都是碎片化、变化快、定制化的。

- 运营想看最近某个 AI 项目的趋势
- 产品想知道某个功能是不是被频繁提及
- 管理者想要一份日报或周报
- 社区维护者想知道最近讨论最热的话题是什么

完成任务并不难，但大多数任务并不是一次性的
bubseek 试图提供更稳定和一致的视图

图源：使用 bubseek 生成 bub 项目和 GitHub Trending 图例（2026.03）

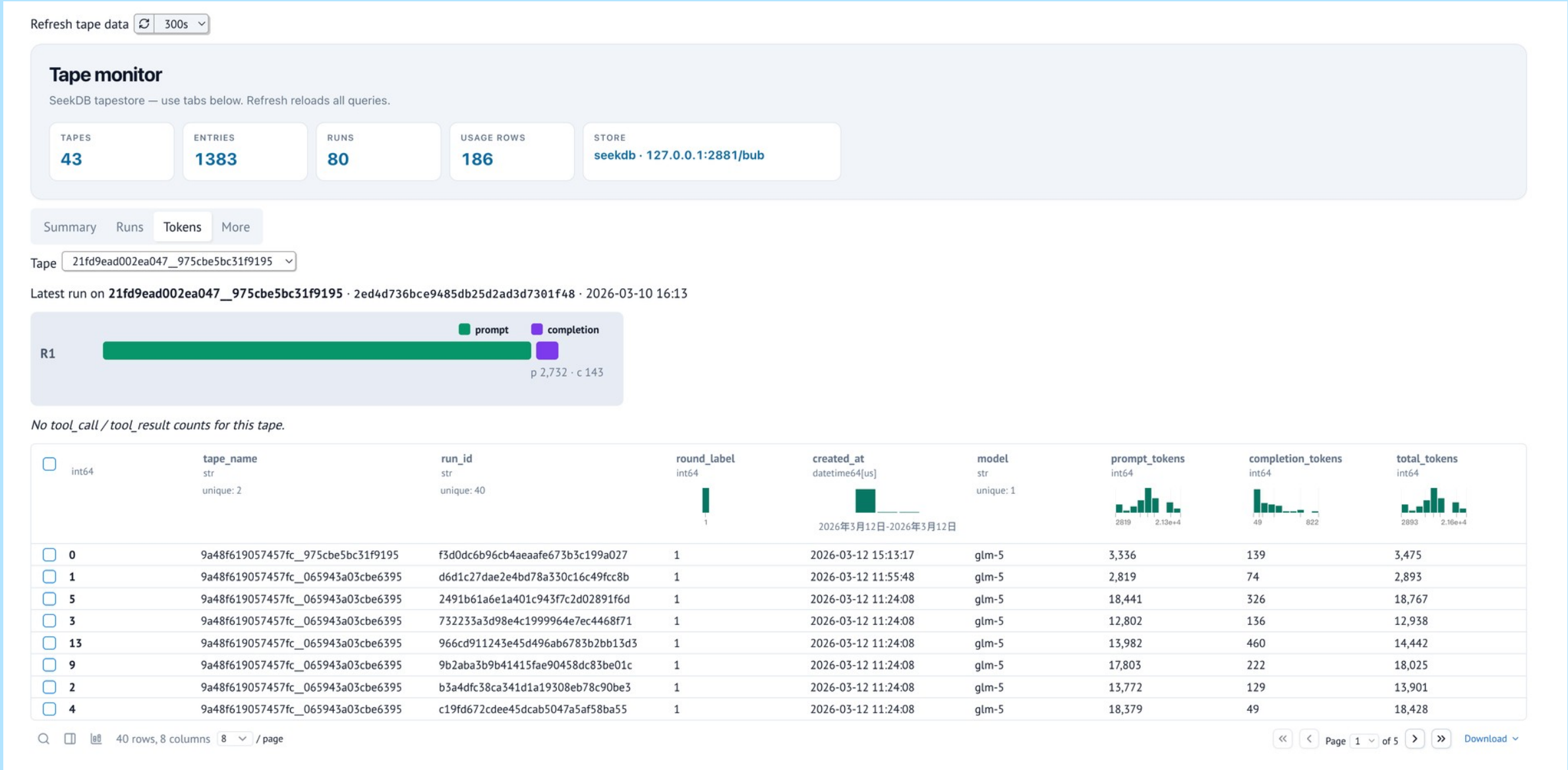


分析 Agent ，其实就是在分析团队

bubseek 更有意思的一点是：它不只分析“外部业务数据”，也分析“ Agent 自己留下来的数据”。

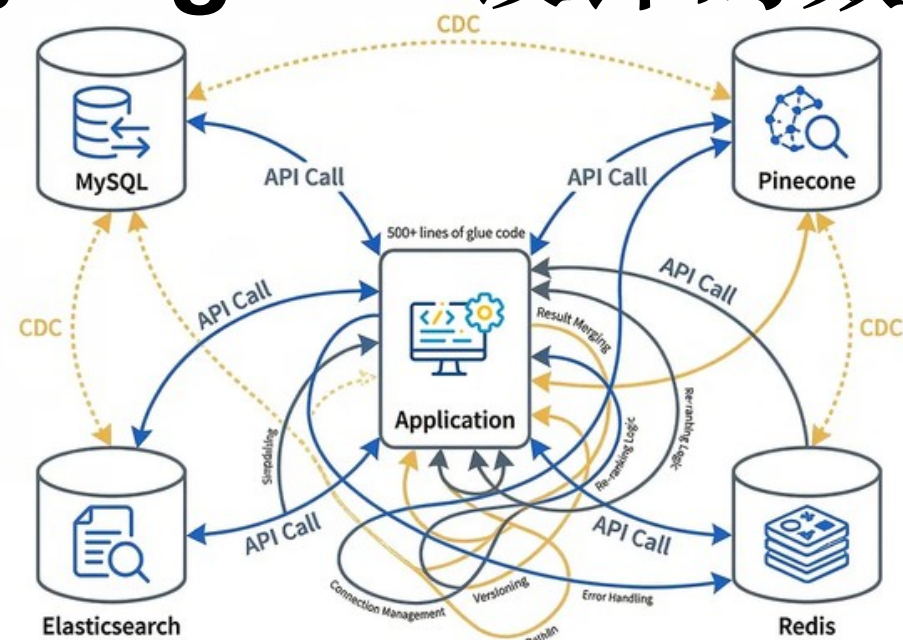
- 哪些类型的任务最容易失败？
- 哪些问题总被反复提及？
- 哪些需求还没有稳定支持？

当团队真的开始使用 Agent 时，Agent 的行为本身就是很有价值的信号。



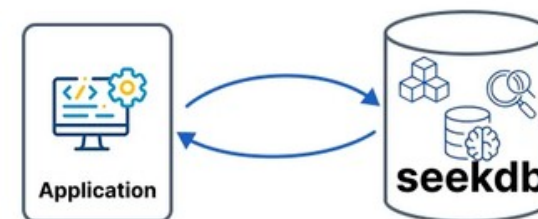
图源：使用 bubseek 中自带的 tape 看板观测运行状态，与 Agent 的信息量一致，无需任何额外的服务（2026.03）

seekdb ，为 Agent 设计的数据库



The Frankenstein Architecture

图源：[reddit 上关于 seekdb 的讨论](#) (2026.03)



The Architecture Base on SeekDB

seekdb 是一个更适合 AI 场景的嵌入式数据库
原生更适合承接：

- 多路检索 (SQL 、 Vector 、 Full-text)
- 融合检索 (设置不同权重，合并检索结果)
- RAG 类工作负载 (向量嵌入)
- 轻量分析与二次消费 (结构化 / 半结构化)

对 bubseek 来说，这意味着：

- 包括 Tape 在内的各种数据可以统一存储
- 后续可以继续问、继续搜、继续分析
- 不需要非常重的部署前提，也不依赖很复杂的环境
- 甚至，无缝扩展到 OceanBase

Part 04

让一切自然发生



没有排期，只有迭代

我们不想再造一个大而全的平台，
而是想让 Agent 时代的数据消费变得更自然，灵活。

提需求、等排期、做接入、上线报表、再修改，
很多数据可能会永远来不及被看见。

所以 bubseek 想做的是另一种节奏：
不强调大而全，
不强调一次性设计完整，
而是让数据消费变成一个自然的迭代过程。



Summary

- **Setosa** is linearly separable (small petals).
- **Versicolor** and **Virginica** overlap; classification is harder but still feasible.

图源：使用 bubseek 迭代分析开源的鸢尾花数据集（2026.03）



欢迎关注



- <https://www.oceanbase.com/>
- <https://github.com/oceanbase/seekdb>
- <https://github.com/ob-labs/bubseek>
- <https://github.com/bubbuild/bub/>
- <https://tape.systems/>



Thanks.

